Fit and Run a Model

Memory, Quantization, Local Models and Serving

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

November 11, 2026

Table of Contents

1. Motivation: from a model in a script to a model many people can call; the map of the lecture
2. What a model needs: small models and GPU memory; prefill and decode
3. The KV cache: repeated work removed, its memory computed by hand, grouped-query attention
4. Quantization: fewer bits per weight, by hand and measured; formats; what fits on a T4
5. Decoding speed and mixture of experts: bytes read per token; total and active parameters
6. Faster answers: long prompts, prefix caching, streaming, speculative decoding
7. Running a model locally and serving it: why a Python script is not a service; GGUF files; Ollama; continuous batching, PagedAttention and vLLM; which to use
8. Training memory: why a training step needs about four times the memory of the weights
9. Summary and references

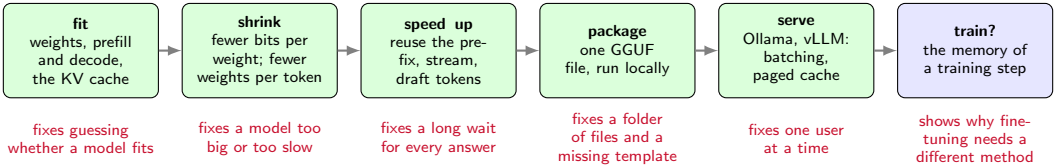
From a Model in a Script to a Model People Use

problem	measured or computed	the fix in this lecture
memory	Qwen3-8B in 16-bit needs 16.4 GB for its weights alone; a T4 has 16 GB	fewer bits per weight
speed	every new token reads all the weights: 0.1 s per token for Qwen3-0.6B on a CPU	the KV cache, smaller weights, experts
waiting	an 8,192-token prompt: 13 s before the first token, on a CPU	reuse the prefix, stream, draft tokens
many users	a Python loop gave 3.3 tokens per second; the same model batched gave 112	a server: Ollama, vLLM
shipping	a model is a folder of configuration, tokenizer and weight files	one GGUF file
training	a training step needs about 16 bytes per parameter: 9.5 GB for Qwen3-0.6B	train a small add-on

Qwen3 parameter counts from the model configurations; speeds on 16 CPU cores. Computed for this course.

- A prompt that works in a script is half an application. The other half is a model that fits the hardware, answers fast, and can be reached by many users at once. Every number in the table comes back later, with its fix measured.

The Map: Fit, Shrink, Speed Up, Package, Serve



- Each step fixes a problem the step before leaves. Red: the problem it fixes.
- Throughout: the Qwen3 family from 0.6B to 8B parameters, and one T4 GPU with 16 GB of memory.

Learning Outcomes

By the end of this lecture you should be able to:

- **Compute** the memory of a model's weights and KV cache by hand; **quantize** a row of weights by hand and **plan** what fits on a T4.
- **Estimate** decoding speed from memory bandwidth, and **explain** why a mixture of experts needs the memory of all its experts but the work of a few.
- **Speed up** answers with prefix caching, streaming and speculative decoding.
- **Explain** what a GGUF file contains, and **run** a local model through Ollama's OpenAI-compatible API.
- **Explain** why a server batches requests and pages the KV cache, and **choose** between a Python script, Ollama and vLLM.
- **Estimate** the memory of a training step, and **explain** why full fine-tuning does not fit where inference does.
- **Prerequisite knowledge:** decoder-only transformers and multi-head attention; chat templates and decoding; prompting and structured output; PyTorch and Hugging Face `transformers`; training with AdamW.

Why Run a Small Model Yourself

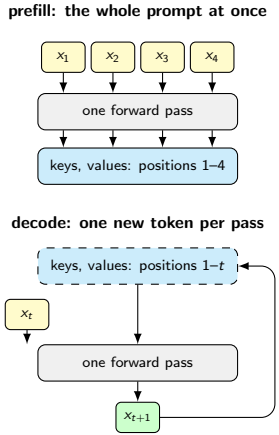
- **The problem with an API for every request:** you pay per token, wait for the network, send your users' data to another company, and the provider can change or retire the model.
- **A small open model on your own GPU:** a fixed cost, data that stays on your machines, a model that never changes unless you change it, and weights you can fine-tune.
- The price: less knowledge and weaker reasoning. Small models do well on narrow, repeated tasks: labels, extraction, routing, short answers from supplied text.
- The question for the rest of the lecture: which model fits on one T4 with 16 GB, and how fast does it answer?

model	parameters	16-bit	Ollama file
Qwen3-0.6B	0.60 B	1.2 GB	0.5 GB
Qwen3-1.7B	1.72 B	3.4 GB	1.4 GB
Qwen3-4B	4.02 B	8.0 GB	2.5 GB
Qwen3-8B	8.19 B	16.4 GB	5.2 GB

Parameters counted from each model's configuration. 16-bit: 2 bytes per parameter. Ollama file: the default download, about 5 bits per parameter. Computed for this course; sizes from ollama.com/library/qwen3.

- **Memory for the weights** = parameters × bytes per parameter. Qwen3-8B in 16-bit is 16.4 GB: more than the whole T4.

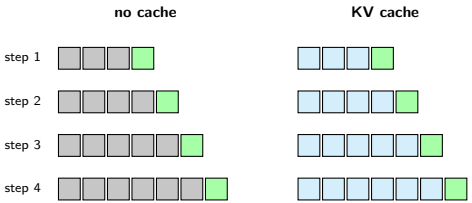
Generating Text Has Two Phases: Prefill, Then Decode



- **Prefill**: all prompt tokens go through the model in one pass, as in training. Large matrix products keep the GPU busy.
- **Decode**: one token per pass, each pass needs the token before it. Every pass reads all the weights from memory to produce one token, so memory bandwidth sets the speed.
- Two numbers a user feels: **time to first token** (TTFT), mostly prefill, and **time per output token** (TPOT), the decode step.
- Latency of one reply $\approx$ TTFT + TPOT $\times$ output tokens. With a TPOT of 0.1 s, a 200-token answer takes 20 s, and a 1,500-token thinking chain 150 s.

0.1 s per token: Qwen3-0.6B on 16 CPU cores with a 512-token prompt. Computed for this course.

Decoding Without a Cache Repeats Work



A 3-token prompt, then 4 decode steps. Green: the new token's keys and values. Grey: recomputed. Blue: read from memory.

- **The problem:** to write token $t+1$, attention needs the keys and values of every earlier token, in every layer. Computed from scratch, step t repeats all the work of step $t-1$.
- **The fix:** the **KV cache**. Keep each token's keys and values once computed; each step computes them for the new token only and reads the rest. A token's keys and values never change, because the causal mask stops it seeing later tokens.
- By hand, 64 new tokens after a 2,048-token prompt: without a cache the model processes $2{,}048 + 2{,}049 + \cdots + 2{,}111 = 133{,}088$ token positions; with it, $2{,}048 + 63 = 2{,}111$.
- `generate` uses the cache by default (`use_cache=True`).

The KV Cache, Measured

prompt tokens	no cache	KV cache	speed-up
128	0.24 s	0.089 s	2.7×
512	0.58 s	0.099 s	5.9×
2,048	2.17 s	0.140 s	15.6×

Seconds per generated token, prefill included: Qwen3-0.6B, 32-bit weights, 16 CPU cores, 64 new tokens after the first words of IMDb reviews, greedy decoding. Computed for this course.

- Without the cache, the time per token grows with everything before it: 2,048 tokens of context make each new token 9 times slower than 128.
- With the cache it barely grows: from 0.089 s to 0.140 s.
- The saving grows with the conversation: long chats, long documents and thinking chains all depend on it.
- **The cost:** the cache lives in GPU memory, next to the weights.

The Cache Costs Memory: Computed by Hand

$$\text{bytes per token} = \underbrace{2}_{K,\,V} \times \text{layers} \times \text{KV heads} \times \text{head size} \times \text{bytes per number}$$

By hand, Qwen3-0.6B in 16-bit:

$2 \times 28 \times 8 \times 128 \times 2 = 114{,}688$ bytes ≈ 115 KB per token.

At its full 32,768-token window: $114{,}688 \times 32{,}768 \approx 3.8$ GB, three times the 1.2 GB of weights.

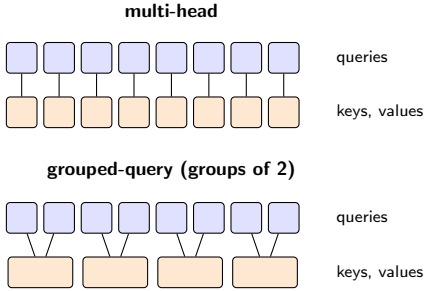
model	per token	8,192 tokens	32,768
Qwen3-0.6B, 1.7B	115 KB	0.9 GB	3.8 GB
Qwen3-4B, 8B	147 KB	1.2 GB	4.8 GB

16-bit cache; layers, KV heads and head size read from each model's configuration. Computed for this course.

- The cache grows with tokens *and* with users: each conversation has its own. Ten users with 8,192 tokens each on Qwen3-4B: 12 GB.

Grouped-Query Attention Shrinks the Cache

- **The problem:** with one key head and one value head per query head, Qwen3-0.6B's 16 heads would need 229 KB per token, twice the cache.
- **The fix: grouped-query attention** (GQA) keeps every query head but lets a group of them share one key head and one value head. Ainslie et al. report "quality close to multi-head attention" with much of the speed of a single shared key-value head.
- Qwen3-0.6B: 16 query heads share 8 key-value heads, half the cache. Qwen3-4B: 32 share 8, a quarter.
- It is fixed when the model is trained: when choosing a model for long inputs, compare KV heads, not only parameters.



Blue: query heads; orange: key-value heads, the part that is cached. Redrawn after Ainslie et al. (2023), Figure 2.

Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," EMNLP 2023, arXiv:2305.13245v3.

What Fills GPU Memory: Weights, Then the Cache

on one T4 (16 GB), 16-bit	weights	cache	fits?
Qwen3-4B, 1 user, 8K tokens	8.0 GB	1.2 GB	yes
Qwen3-4B, 8 users, 8K tokens	8.0 GB	9.7 GB	no
Qwen3-8B, no tokens yet	16.4 GB	0	no

8K: 8,192 tokens per conversation. Leave 1 to 2 GB more for activations and the CUDA libraries.

- GPU memory holds the **weights**, the **KV cache** of every active conversation, and working memory for one forward pass.
- Whatever the weights leave free decides how many users and how many tokens you can serve.
- **The problem:** in 16-bit, an 8B model does not load at all, and a 4B model leaves room for only a few users.
- **The fix:** store each weight in fewer bits.

Store Fewer Bits per Weight

precision	bytes per weight	Qwen3-4B	Qwen3-8B
32-bit	4	16.1 GB	32.8 GB
16-bit	2	8.0 GB	16.4 GB
8-bit	1	4.0 GB	8.2 GB
4-bit	0.5	2.0 GB	4.1 GB

Weights only, before the small extra cost of scales. Computed for this course.

- **Quantization**: store each weight as a small integer, plus one shared **scale** that turns the integers back into real numbers.
- At run time the integers are converted back (**dequantized**) and multiplied in 16-bit. Memory falls 2 to 4 times; with fast kernels, decode speeds up too, because each step reads fewer bytes.
- **The question:** rounding to 256 or 16 levels loses information. How much, and does the model notice?

Quantization by Hand: One Row of Weights

$w = (0.12,\ -0.50,\ 0.33,\ 0.02)$

8-bit, integers $-127 \ldots 127$:
scale $s = \max|w|/127 = 0.50/127 = 0.003937$
$q = \mathrm{round}(w/s) = (30,\ -127,\ 84,\ 5)$
back: $q \times s = (0.1181,\ -0.5000,\ 0.3307,\ 0.0197)$

4-bit, integers $-7 \ldots 7$:
$s = 0.50/7 = 0.0714$
$q = (2,\ -7,\ 5,\ 0)$
back: $(0.143,\ -0.500,\ 0.357,\ 0.000)$

- **Absmax quantization**: the largest weight sets the scale, so it maps to the largest integer; the others round to the nearest step.
- 8-bit: every weight within 0.002 of its value. 4-bit: errors up to 0.027, and 0.02 becomes exactly 0.
- Stored: 4 small integers and one 16-bit scale. The more weights share a scale, the cheaper the scale.
- Also common: an offset (**zero point**) so that the integers cover $[\min w, \max w]$ instead of $[-\max|w|, \max|w|]$.

One Large Weight Ruins the Scale: Quantize in Small Groups

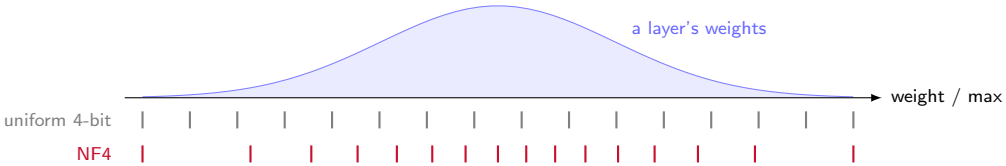
$w = (0.03,\ -0.05,\ 0.02,\ 0.04 \mid 2.40,\ -0.01,\ 0.06,\ -0.03)$

One 4-bit scale for the row: $s = 2.40/7 = 0.343$
back: $(0,\ 0,\ 0,\ 0 \mid 2.40,\ 0,\ 0,\ 0)$: seven of eight weights lost.

One scale per group of 4:
first group: $s = 0.05/7 = 0.00714$
back: $(0.029,\ -0.050,\ 0.021,\ 0.043)$
second group: still $(2.40,\ 0,\ 0,\ 0)$

- **The problem:** trained models have a few weights far larger than the rest. One scale per row lets them set the step size, and the small weights round to zero.
- **The fix: group-wise quantization**: one scale per group of 32 to 128 neighbouring weights. An outlier now damages only its own group.
- The cost: one 16-bit scale per 128 weights is $16/128 = 0.125$ extra bits per weight. "4-bit" formats store about 4.1 to 5 bits per weight.

NF4: Put the 16 Levels Where the Weights Are



- **The problem:** a trained layer's weights are bell-shaped: most near zero, few large. Evenly spaced levels spend codes on the tails where there are few weights.
- **The fix: NF4** (4-bit NormalFloat, from QLoRA) places its 16 levels at the quantiles of a normal distribution, so each level is used about equally often; QLoRA calls it "information theoretically optimal for normally distributed weights".
- It is the 4-bit type of the `bitsandbytes` library, one scale per 64 weights.

Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," arXiv:2305.14314v1; NF4 levels from the `bitsandbytes` source. Bell curve illustrative.

Quantization Quality, Measured

Qwen3-1.7B	bits	size	perplexity	En	Ar
16-bit	16	3.4 GB	20.4	90%	88%
8-bit, per row	8.0	2.0 GB	20.2	90%	87%
4-bit, groups of 128	4.25	1.4 GB	24.7	89%	78%
NF4, groups of 64	4.25	1.4 GB	22.6	92%	87%
3-bit, groups of 128	3.25	1.2 GB	69.4	46%	5%
2-bit, groups of 128	2.25	1.0 GB	4×10^6	0%	0%

Every linear layer rounded and converted back; embeddings kept in 16-bit (size includes them). Bits: per quantized weight, scales included. Perplexity: $e^{\text{average next-token loss}}$ on 20,480 WikiText-2 test tokens, lower is better. En, Ar: intent accuracy on 100 MASSIVE requests each, 10 examples as chat turns. Computed for this course.

- **8-bit**: no measurable loss at half the memory.
- **4-bit**: NF4 costs little. Plain 4-bit rounding costs more, and Arabic shows it first: 88% to 78%.
- **3-bit**: English accuracy halves and Arabic collapses to 5%. **2-bit**: the model is broken.
- Test every language you serve: perplexity on English text hides the Arabic loss.

Fewer Bits, Faster Decoding

GGUF type	bits	size	prefill	decode
F16	16.0	15.0 GiB	923	29
Q8_0	8.5	8.0 GiB	865	51
Q4_K_M	4.9	4.6 GiB	822	72
Q2_K	3.2	3.0 GiB	784	80

Llama-3.1-8B in `llama.cpp`. Prefill: prompt tokens per second for a 512-token prompt; decode: generated tokens per second, 128 tokens. Bits: per weight, scales included. Hardware not stated in the source. From the `llama.cpp` quantization README, github.com/ggml-org/llama.cpp.

- Decode reads every weight once per token. Half the bytes, nearly twice the tokens per second: 29 at 16 bits, 72 at about 5.
- Prefill hardly changes: it is limited by arithmetic, not by reading weights.
- Below 4 bits the speed gain is small and the quality loss is large.
- Not every library gains speed: `bitsandbytes` converts weights back on the fly, and for a small model on a T4 its 4-bit can be slower than 16-bit. Its point is memory.

Quantized Formats You Will Meet

format	bits	how it is made	runs on	used with
`bitsandbytes` 8-bit, NF4	8, 4	when the model is loaded; no data needed	NVIDIA GPUs	`transformers`; fine-tuning with QLoRA
GGUF (`Q8_0`, `Q4_K_M`, ...)	2 to 8	converted once with the `llama.cpp` tools; ready-made files on the Hub	CPUs, Apple chips, GPUs	`llama.cpp`, Ollama
GPTQ, AWQ	3, 4	once, using a small set of example texts to decide how to round	NVIDIA GPUs	vLLM, `transformers`

- **GPTQ** and **AWQ** look at real activations so that rounding errors land where they matter least; at 3 to 4 bits they lose less than plain rounding.
- `Q4_K_M`, Ollama's default, mixes types across tensors, some above 4 bits: Qwen3-8B's file is 5.2 GB, about 5 bits per weight.

Dettmers et al., LLM.int8(), arXiv:2208.07339v2; Frantar et al., GPTQ, arXiv:2210.17323v2; Lin et al., AWQ, arXiv:2306.00978v6; github.com/ggml-org/llama.cpp.

Loading a Quantized Model

Hugging Face, on a GPU: NF4 at load time

```
import torch
from transformers import (
    AutoModelForCausalLM, BitsAndBytesConfig)

cfg = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16)
model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen3-8B",
    quantization_config=cfg,
    device_map="auto")
print(model.get_memory_footprint() / 1e9)
```

Ollama, on a laptop: a ready-made GGUF

```
ollama pull qwen3:8b        # Q4_K_M
ollama pull qwen3:8b-q8_0   # 8-bit
ollama run qwen3:8b
```

- `bnb_4bit_compute_dtype`: the type used for the multiplications after dequantizing. The T4 has no fast `bfloat16`, so use `float16`.
- `get_memory_footprint` returns bytes: check the model fits before serving it.
- The tag after the colon picks size and quantization: 5.2 GB at `Q4_K_M`, 8.9 GB at 8 bits.

Hugging Face, "bitsandbytes," huggingface.co/docs/transformers; ollama.com/library/qwen3/tags.

What Fits on a T4

model	weights 16-bit	weights 8-bit	weights 4-bit	8K-token conversations
Qwen3-1.7B	3.4	1.7	0.9	13
Qwen3-4B	8.0	4.0	2.0	9
Qwen3-8B	16.4	8.2	4.1	8
Qwen3-14B	29.5	14.8	7.4	4

GB. Red: does not fit. Conversations: how many 8,192-token KV caches fit beside the 4-bit weights in 16 GB, keeping 2 GB spare. Computed for this course.

- 8-bit is nearly free in quality; 4-bit with groups costs a little; below 4 bits, quality falls fast.
- For a fixed memory budget, a larger model at 4 bits usually beats a smaller one at 16 bits: Dettmers and Zettlemoyer found "4-bit precision is almost universally optimal for total model bits and zero-shot accuracy".
- Check on your own test set: quantization hurts some tasks and languages more than others.

Dettmers and Zettlemoyer, ICML 2023, arXiv:2212.09720v2.

How Fast Can a Model Decode? Computed by Hand

$$\text{tokens per second} \leq \frac{\text{memory bandwidth}}{\text{bytes read per token}}$$

One request at a time; the bytes read per token are about the size of the weights. A T4 reads 320 GB/s.

	16-bit		4-bit	
model	**GB**	**at most**	**GB**	**at most**
Qwen3-1.7B	3.4	94/s	0.9	356/s
Qwen3-4B	8.0	40/s	2.0	160/s
Qwen3-8B	16.4	no fit	4.1	78/s

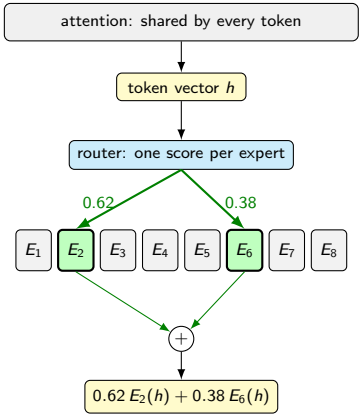
T4 tokens per second, upper bounds: 320/GB. Computed for this course; bandwidth from NVIDIA's T4 datasheet.

on 16 CPU cores	**tokens/s**	**GB read/s**
Qwen3-0.6B, 2.4 GB	11.9	28
Qwen3-1.7B, 6.9 GB	5.0	34
Qwen3-4B, 16.1 GB	2.1	34

32-bit weights, one request, greedy. GB read/s = weight GB × tokens/s. Computed for this course.

- From 0.6B to 4B: 6.7 times the bytes, 5.6 times slower. The read rate stays near 30–34 GB/s whatever the size: the speed of memory sets the pace.
- Halve the bytes, double the ceiling: why quantization speeds up decoding, and why a batch helps: one read of the weights serves every request in it.

Mixture of Experts: More Parameters, Same Work



- **The problem:** in a dense model every token passes through every weight. More parameters, more knowledge, but also more work and more memory reads for every token.
- **The fix:** replace the feed-forward block of each layer with many smaller feed-forward blocks, the **experts**, and a **router** that sends each token to a few of them.
- The router is one linear layer: a score per expert, keep the top k, mix their outputs with softmax weights.
- In the picture, six of the eight experts are skipped for this token. Attention and embeddings stay shared; each token picks its own experts, in every layer separately.

Shazeer et al., 2017, arXiv:1701.06538v1; top-1 routing: Fedus, Zoph and Shazeer, 2022, arXiv:2101.03961v3; top-2 of 8: Mixtral, arXiv:2401.04088v1.

A Router by Hand: One Token, Four Experts

	E_1	E_2	E_3	E_4
router score $h \cdot w_i$	2.0	0.5	1.5	-1.0
e^{score}	7.39	1.65	4.48	0.37
softmax (sum 13.89)	0.53	0.12	0.32	0.03
top 2 kept	**0.53**	–	**0.32**	–
weights, renormalized	**0.62**	0	**0.38**	0

output $= 0.62\, E_1(h) + 0.38\, E_3(h)$

E_2 and E_4 are never run for this token: their weights are not read.

- **Step 1:** multiply the token vector by one weight vector per expert: four scores.
- **Step 2:** softmax, keep the $k = 2$ largest, divide by their sum so the weights add to 1.
- **Step 3:** run only the chosen experts and add their outputs with these weights.
- **A training problem:** a router can learn to send almost every token to the same few experts, and the rest are wasted. The fix in training: an extra loss that rewards even use of the experts (Switch Transformers).

Total and Active Parameters, Counted by Hand

model	experts	per token	total	active
OLMoE-1B-7B	64	8	6.9 B	1.3 B
Qwen3-30B-A3B	128	8	30.5 B	3.35 B
Qwen3-235B-A22B	128	8	235.1 B	22.2 B
gpt-oss-20b	32	4	21 B	3.6 B
Mixtral 8x7B	8	2	47 B	13 B
DeepSeek-V3	256+1	8+1	671 B	37 B

Top three: counted for this course from each model's `config.json` and weight files. Bottom three: as reported by the model's authors; DeepSeek-V3 adds one shared expert that every token uses.

Qwen3-30B-A3B by hand

one expert: $3 \times 2048 \times 768 = 4.7\,\text{M}$
48 layers $\times$ 128 experts: 29.0 B in experts
unused per token: $48 \times 120 \times 4.7\,\text{M} = 27.18\,\text{B}$
active: $30.53 - 27.18 = 3.35\,\text{B}$

- **Total** decides memory: every expert must be loaded, because the next token may need it.
- **Active** decides the work per token: Qwen3-30B-A3B reads about as many weights per token as a dense 3B model.

Memory Follows Total, Speed Follows Active, Measured

model	total	active	memory	tokens/s
OLMo-1B, dense	1.3 B	1.3 B	5.1 GB	9.7
OLMoE-1B-7B	6.9 B	1.3 B	27.7 GB	**7.3**
OLMo-7B, dense	6.9 B	6.9 B	27.6 GB	2.3

32-bit weights on 16 CPU cores, one request, greedy decoding. All three from the same lab (Ai2). Computed for this course.

- The mixture of experts needs the memory of the dense 7B model, and decodes 3.2 times faster than it: close to the dense 1B model, whose size matches its active part.

- **On a T4:** Qwen3-30B-A3B in 4-bit is a 19 GB file: it does not fit in 16 GB, even though it answers at the speed of a 3B model. gpt-oss-20b was released with its experts in 4-bit and is a 14 GB file.
- **Too big for the GPU:** `llama.cpp` can keep the experts in CPU memory and the rest on the GPU, with `--cpu-moe` or `--n-cpu-moe N`.
- **Choose a mixture of experts** when memory is plentiful and speed matters: a server with large GPUs, a laptop with 32 GB or more. **Choose a dense model** when memory is the limit, as on a T4.

Long Prompts Delay the First Token, Measured

prompt tokens	first token	per output token
128	0.21	0.085
512	0.53	0.093
1,024	1.00	0.096
2,048	1.99	0.105
4,096	4.99	0.125
8,192	13.1	0.174

Qwen3-0.6B, 32-bit, 16 CPU cores; seconds. First token: the prefill of the whole prompt. Computed for this course.

- The first token waits for the prefill of the whole prompt. Up to 2,048 tokens, twice the prompt is twice the wait; beyond that it grows faster, because attention compares every pair of tokens: 13 s at 8,192.
- Each output token also slows down, from 0.085 s to 0.174 s: every step reads a larger KV cache.
- **The problem it leaves:** a prompt with instructions, a list of 60 labels and 120 examples is 2,151 tokens, and the same 2,151 tokens are prefilled again for every request.

Reuse a Fixed Prefix: Compute It Once, Copy Its Cache

```
import copy
from transformers import DynamicCache

# the fixed part: instructions + examples
prefix = tok(fixed_part, return_tensors="pt").
    input_ids
with torch.no_grad():
    cache = model(prefix,
        past_key_values=DynamicCache()).
            past_key_values

for request in requests:
    new = tok(request, return_tensors="pt").
        input_ids
    ids = torch.cat([prefix, new], dim=1)
    out = model.generate(ids, max_new_tokens=8,
        past_key_values=copy.deepcopy(cache))
```

- **The fix:** prefill the fixed part once and keep its KV cache. Each request starts from a copy and prefills only its own new tokens.
- The copy matters: generation appends to the cache, and the next request needs it clean.
- **The condition:** the prefix must be identical, token for token. A date or a user name in the system prompt breaks the reuse from that token on. Put the fixed part first and the changing part last.
- vLLM does this for every request with `--enable-prefix-caching`.

Prefix Caching, Measured

	seconds
prefill the fixed 2,151 tokens, once	7.3
first token, whole prompt every time	7.1
first token, copied cache + 16 new tokens	**0.41**
same answer as without the cache	8 of 8

Qwen3-1.7B, 32-bit, 16 CPU cores; voice-assistant requests, with 60 labels and 120 examples in the system message; 8 requests, median of the fastest of three runs. Computed for this course.

- The first token came 17 times sooner: each request prefilled 16 tokens instead of 2,167.
- The answers did not change: the cache holds exactly what the prefill would have computed.
- The cache costs memory: 115 KB per token for Qwen3-1.7B in 16-bit, so the 2,151-token prefix holds 0.25 GB. One copy per request in flight.

Stream the Reply: Show the First Token at Once

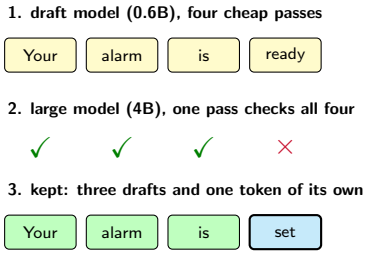
```
# transformers: a streamer and a thread
from threading import Thread
from transformers import TextIteratorStreamer
streamer = TextIteratorStreamer(tok,
    skip_prompt=True, skip_special_tokens=True)
Thread(target=model.generate, kwargs=dict(**
    inputs,
    streamer=streamer, max_new_tokens=300)).start
        ()
for piece in streamer:
    print(piece, end="", flush=True)

# Ollama or vLLM: the same OpenAI client
stream = client.chat.completions.create(
    model=name, messages=messages, stream=True)
for chunk in stream:
    print(chunk.choices[0].delta.content or "",
        end="")
```

- **The problem:** a 200-token reply from Qwen3-0.6B on 16 CPU cores took 16.8 s, but its first token was ready after 0.1 s. Without streaming, the user stares at an empty box for 16.8 s.
- **The fix:** send each token as soon as it is made. The total time is the same; the user starts reading after the first token.
- Every chat interface does this. Do not stream when the answer must be parsed whole, such as JSON for another program.

Timings: Qwen3-0.6B, 32-bit, a 29-token prompt. Computed for this course.

Speculative Decoding: Draft Small, Check Large



Four tokens for one pass of the large model.

- **The problem:** one pass of the large model per token, and each pass reads all its weights to make one token.
- **The fix:** a small model with the same tokenizer guesses the next few tokens. The large model checks them all in one pass, as in prefill, and keeps them up to the first disagreement, plus its own token there.
- The output is the large model's own output: the drafts only decide how many tokens each pass produces.
- **Prompt lookup:** when the answer copies the prompt (rewriting, extraction), take the drafts from the prompt itself. No second model.

Leviathan, Kalman and Matias, ICML 2023, arXiv:2211.17192v2; Chen et al., 2023, arXiv:2302.01318v1; prompt lookup: Saxena, 2023, github.

Speculative Decoding, Measured

task	drafts from	tokens/s	per pass	same
free answer	none	1.98	1.0	–
	Qwen3-0.6B	2.90	2.2	4/4
	the prompt	2.02	1.0	4/4
rewrite	none	2.01	1.0	–
	Qwen3-0.6B	5.76	**10.7**	4/4
	the prompt	**5.99**	3.8	4/4

Qwen3-4B, 32-bit, 16 CPU cores, greedy, up to 64 new tokens, 4 prompts per task. Tokens/s: the fastest of three runs. Per pass: new tokens per pass of the 4B model. Same: identical to plain decoding. Computed for this course.

```
draft = AutoModelForCausalLM.from_pretrained("
    Qwen/Qwen3-0.6B")
model.generate(**inputs, assistant_model=draft)
model.generate(**inputs, prompt_lookup_num_tokens
    =10)
```

vLLM: `--speculative-config` with `"method": "ngram"` (prompt lookup) or `"draft_model"`.

- Every answer was identical to plain decoding: the 4B model decides every token.
- **Rewrite:** the answer copies the review, so most drafts are right. 10.7 tokens per pass with the 0.6B draft, 2.9 times faster; prompt lookup, with no second model, 3.0 times.
- **Free answer:** drafts are often wrong. The 0.6B draft gave 1.5 times; prompt lookup found nothing to copy and gained nothing.

A Python Script Is Not a Service, Measured

```
for request in requests:       # one at a time
    ids = tok(render(request),
              return_tensors="pt")
    out = model.generate(**ids,
                         max_new_tokens=64)
```

32 requests, 64 new tokens each	tokens per second
one request at a time	3.3
batches of 8	67
one batch of 32	**112**
7 one-word answers in one batch	
alone	2.2 s
with one long story in the batch	66.6 s

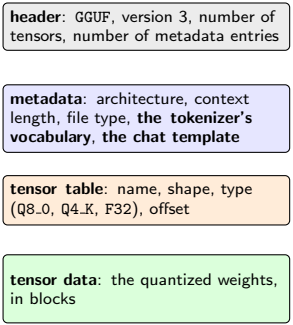
Qwen3-0.6B with `transformers generate`, 16 CPU cores, greedy.

In the mixed batch, 13% of the generated positions were real tokens; the rest was padding after the short answers ended.

Computed for this course.

- **The problem:** a script that calls `generate` is a library inside one program. Every program loads its own copy of the model, nobody else can reach it, and requests are answered one call at a time.
- One at a time wastes the hardware: each decode step reads all the weights for a single token. Batching 32 requests gave 34 times the tokens per second.
- A plain batch waits for its longest member: seven one-word answers took 30 times longer next to one story.
- **The fix:** a model server. One process holds the model, answers HTTP requests from any program, and batches them as they arrive.

GGUF: One File With Everything a Runner Needs



- **The problem:** a Hugging Face model is a folder: a configuration, tokenizer files, the chat template and 16-bit weights. A program on a laptop needs all of it, and smaller.
- **The fix:** GGUF, "a file format for storing models for inference with GGML and executors based on GGML", built for "single-file deployment".
- "Unlike tensor-only file formats like safetensors . . . GGUF encodes both the tensors and a standardized set of metadata": the tokenizer and the chat template travel with the weights, so the right template is always applied.
- The weights are usually quantized in blocks, as in group quantization. Programs that read GGUF: `llama.cpp`, Ollama, LM Studio.

GGML, "GGUF" specification, github.com/ggml-org/ggml; Hugging Face, "GGUF," huggingface.co/docs/hub/gguf.

Inside a Real GGUF File, Measured

```
import struct
f = open(path, "rb")  # Qwen3-0.6B-Q8_0.gguf
magic = f.read(4)
version, = struct.unpack("<I", f.read(4))
n_tensors, n_meta = struct.unpack(
    "<QQ", f.read(16))
klen, = struct.unpack("<Q", f.read(8))
key = f.read(klen).decode()
print(magic, version, n_tensors, n_meta)
print(key)
# b'GGUF' 3 310 28
# general.architecture
```

The header, read with Python's `struct` from the layout in the GGUF specification: no `llama.cpp` needed.

read from the file	value
architecture	qwen3
blocks	28
key-value heads	8
context length	40,960
tokenizer vocabulary	151,936 tokens
chat template	4,100 characters
tensors	197 Q8_0, 113 F32
file size	639 MB
16-bit safetensors	1,503 MB

`Qwen/Qwen3-0.6B-GGUF`, all 28 metadata entries read for this course.

- Everything needed to run the model is inside. The F32 tensors are the small normalization weights, kept exact.
- 8-bit blocks make the file 43% of the 16-bit one.

Where GGUF Files Come From

Ready-made, from the Hugging Face Hub

```
from huggingface_hub import hf_hub_download
path = hf_hub_download("Qwen/Qwen3-0.6B-GGUF",
                       "Qwen3-0.6B-Q8_0.gguf")
```

Your own model, with the `llama.cpp` tools

```
python convert_hf_to_gguf.py \
  --outfile qwen3-1.7b-bf16.gguf \
  --outtype bf16 --remote Qwen/Qwen3-1.7B
./build/bin/llama-quantize \
  qwen3-1.7b-bf16.gguf \
  qwen3-1.7b-Q4_K_M.gguf Q4_K_M
```

Commands in the form of the `llama.cpp` quantize README; for a fine-tuned model, give its local folder instead of `--remote`.

GGUF type	bits per weight
Q8_0	8.5
Q6_K	6.56
Q5_K	5.5
Q4_K	4.5
Q3_K	3.44
Q2_K	2.63

Hugging Face, "GGUF". `Q8_0`: blocks of 32 weights, each block with one 16-bit scale.

- Bits per weight include the scales stored with each block, so a 4-bit type costs about 4.5.
- Files named `Q4_K_M` or `Q4_K_S` mix types across tensors; Ollama's default downloads are `Q4_K_M`.

Ollama: A Model Server on Your Own Machine

```
ollama pull qwen3:1.7b    # 1.4 GB, Q4_K_M
ollama run qwen3:1.7b     # chat in a terminal
```

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:11434/v1/",
    api_key="ollama")  # required, ignored
reply = client.chat.completions.create(
    model="qwen3:1.7b",
    messages=[{"role": "user",
      "content": "wake me up at seven"}])
print(reply.choices[0].message.content)
```

- The simplest model server: `ollama pull` downloads a model once; the Ollama service keeps it loaded and answers every program on the machine.
- **Ollama** downloads quantized GGUF models by name and runs them on a CPU, an Apple chip or a GPU. It "supports a subset of the OpenAI API": code written for OpenAI works by changing `base_url`.
- Your own model: a Modelfile with `FROM ./model.gguf`, then `ollama create`. Quantize it first; Ollama "does not quantize GGUF models during import".

Ollama documentation, "OpenAI compatibility" and "Importing a Model," docs.ollama.com.

Your Own Model in Ollama: A Modelfile

Modelfile

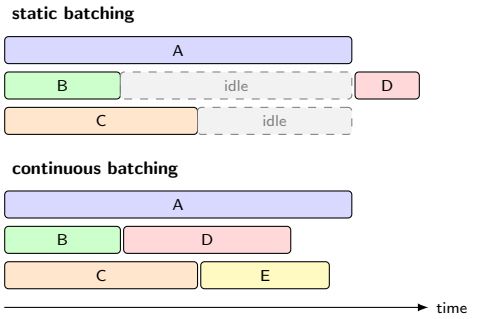
```
FROM ./qwen3-1.7b-Q4_K_M.gguf
PARAMETER temperature 0.7
PARAMETER num_ctx 8192
SYSTEM """You sort requests for a
voice assistant. Answer with the
intent name only."""
```

```
ollama create intent-bot -f Modelfile
ollama run intent-bot
```

- `FROM` "defines the base model to use when creating a model": a model name, or a GGUF file.
- `PARAMETER` sets decoding and runtime values. `num_ctx` "sets the size of the context window" (default 2048); `temperature` defaults to 0.8.
- `SYSTEM` "specifies the system message to be used in the template"; the template itself comes from the GGUF file.
- Ollama "does not quantize GGUF models during import": quantize first, with `llama-quantize`.
- Why run locally: the data never leaves the machine, it works offline, the cost does not grow with use, and the model never changes under you.

Ollama documentation, "Modelfile" and "Importing a Model," docs.ollama.com.

Many Users at Once: Continuous Batching



Each row is one slot in the batch. D and E arrive while A runs.

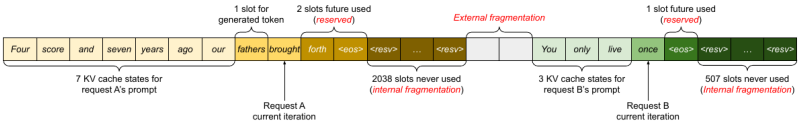
- **The problem:** one decode step reads all the weights to produce one token. Serving users one at a time wastes most of the GPU.
- **Batching**: one step produces the next token for many requests, for nearly the price of one.
- **Static batching** keeps a batch together until its longest request ends: short requests leave idle slots, new ones wait.
- **Continuous batching** reschedules after every decode step: a finished request leaves, a waiting one joins. Orca called it **iteration-level scheduling**.

Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI 2022.

PagedAttention and vLLM: The KV Cache in Pages

- **The problem:** earlier servers reserved each request's KV cache for its maximum length, in one block. Memory sat reserved but unused, and gaps between blocks could not be used.
- **The fix: PagedAttention**. Split the cache into small fixed-size blocks, allocated as tokens arrive, like pages of memory in an operating system. A table maps each request to its blocks.
- **vLLM**, built on it with continuous batching: "improves the throughput of popular LLMs by 2-4× with the same level of latency".

```
pip install vllm
vllm serve Qwen/Qwen3-1.7B
# OpenAI API on port 8000
```



Two requests in an earlier serving system: slots reserved for future tokens, internal and external fragmentation. Kwon et al., SOSP 2023, arXiv:2309.06180v1, Figure 3.

vLLM in Practice: Serve, Configure, Call

```
pip install vllm
vllm serve Qwen/Qwen3-1.7B \
  --max-model-len 8192 \
  --gpu-memory-utilization 0.90 \
  --enable-prefix-caching
```

```
from openai import OpenAI
client = OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="unused")
reply = client.chat.completions.create(
    model="Qwen/Qwen3-1.7B",
    messages=[{"role": "user",
               "content": "wake me up at seven
                   "}])
```

- `--max-model-len`: "Model context length (prompt and output)." Lower it to leave more memory for the KV cache.
- `--gpu-memory-utilization`: "The fraction of GPU memory to be used for the model executor"; vLLM fills it with KV-cache blocks.
- `--enable-prefix-caching`: it "caches the KV cache of existing queries", "allowing the new query to skip the computation of the shared part". A long system prompt is computed once.
- `--quantization`: the "method used to quantize the weights", for AWQ or GPTQ files.
- Needs an NVIDIA GPU of "compute capability 7.5 or higher (e.g., T4, ...)".

vLLM documentation, "Engine Arguments," "Automatic Prefix Caching" and "Installation," docs.vllm.ai.

A Python Script, Ollama or vLLM

	Python script (`transformers`)	**Ollama**	**vLLM**
built for	experiments, training, evaluation	one machine, a few users	a GPU server, many users
requests	calls inside one program	an HTTP API; one loaded model shared by every program	an HTTP API with continuous batching
memory	a batch reserves its whole KV cache	quantized GGUF weights	KV cache in pages; prefix caching
model files	Hugging Face weights	GGUF	Hugging Face weights, AWQ, GPTQ
hardware	CPU or GPU	CPU, Apple chips, GPUs	NVIDIA GPUs, from a T4 up
you write	the template, decoding and batching	nothing: `ollama run`	nothing: `vllm serve`

- Develop and evaluate in Python; serve with Ollama on a laptop or vLLM on a GPU. Both speak the OpenAI API: moving between them changes one URL.
- Similar tools: `llama.cpp` itself, LM Studio (a desktop app), SGLang and TensorRT-LLM (GPU servers).

Training Needs Far More Memory Than Running

- **The problem:** a model that runs on a T4 may not train on it. A training step keeps three more things for every weight:
  - its **gradient**, the same size as the weight;
  - AdamW's two running averages of the gradient and of its square, in 32-bit;
  - in mixed precision, a 32-bit master copy of the weight.
- Plus the activations of the batch, kept for the backward pass; they grow with batch size and sequence length.

By hand, bytes per parameter in mixed precision:
2 (16-bit weight) +2 (gradient) +4 (32-bit master copy) +8 (AdamW, two 32-bit values) = 16

model	run (16-bit)	full training
Qwen3-0.6B	1.2 GB	9.5 GB
Qwen3-1.7B	3.4 GB	27.5 GB
Qwen3-4B	8.0 GB	64.4 GB
Qwen3-8B	16.4 GB	131.1 GB

2 and 16 bytes per parameter, before activations. Computed for this course from the parameter counts.

- On a 16 GB T4, only Qwen3-0.6B could be fully fine-tuned, with little room for activations; Qwen3-1.7B, which runs easily, does not train.

Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020, arXiv:1910.02054v3 (the 16-byte count).

Train a Small Add-On Instead, Measured

Qwen3-0.6B, one AdamW step	full	add-on (LoRA)
trained parameters	596.0 M	10.1 M (1.7%)
weights	2.38 GB	2.42 GB
gradients	2.38 GB	0.04 GB
AdamW states	4.77 GB	0.08 GB
total	**9.54 GB**	**2.54 GB**

Exact bytes of the tensors after one step, 32-bit on a CPU ($4 + 4 + 8 = 16$ bytes per trained parameter). LoRA: rank 16 on every linear layer of every block, with `peft`. Activations not included. Computed for this course.

- **The fix:** freeze the model and train a small add-on. **LoRA** (low-rank adaptation) places two thin matrices beside each weight matrix and trains only those.
- Gradients and AdamW states then exist only for 1.7% of the parameters: 121 MB instead of 7.15 GB.
- The frozen weights still fill memory. Stored in 4 bits (NF4), they shrink by four: that combination is **QLoRA**, and it lets a T4 fine-tune models of several billion parameters.

Summary

- **Memory for the weights** is parameters × bytes per parameter: Qwen3-8B in 16-bit (16.4 GB) does not fit a 16 GB T4.
- **Decode is limited by memory:** the KV cache removes repeated work (15.6× faster at 2,048 tokens) and costs 2× layers × KV heads × head size × bytes per token; grouped-query attention shrinks it.
- **Quantization:** 8-bit is nearly free, NF4 costs little, plain 4-bit rounding cost Arabic 10 points, 3-bit collapses. Test every language you serve.
- **Decoding speed** is at most memory bandwidth / bytes read per token. A **mixture of experts** keeps every expert in memory but runs only a few per token: OLMoE-1B-7B needed the memory of a 7B model and decoded 3.2 times faster than one.
- **Faster answers:** a long prompt delays the first token (13 s at 8,192 tokens); reusing the cache of a fixed 2,151-token prefix brought the first token from 7.1 s to 0.41 s; stream the reply; speculative decoding keeps the large model's own output with fewer passes (10.7 tokens per pass and 2.9 times faster on a rewrite).
- **A Python script is not a service:** batching 32 requests gave 34 times the tokens per second of a loop, and a plain batch makes short answers wait for the longest one.
- **GGUF** puts the quantized weights, the tokenizer and the chat template in one file: 639 MB for Qwen3-0.6B in `Q8_0`, against 1.5 GB in 16-bit.
- **Ollama** runs GGUF files locally behind an OpenAI-compatible API; a Modelfile adds your system prompt and settings. **vLLM** serves many users on a GPU with continuous batching, a paged KV cache and prefix caching.
- **Training needs about 16 bytes per parameter:** 9.5 GB for Qwen3-0.6B. Training a small add-on (LoRA) cut gradients and optimizer states from 7.15 GB to 0.12 GB.

References

1. Qwen Team, "Qwen3 Technical Report," 2025. arXiv:2505.09388v1; model cards, huggingface.co/Qwen
2. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," EMNLP 2023. arXiv:2305.13245v3
3. Dettmers, Lewis, Belkada and Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," NeurIPS 2022. arXiv:2208.07339v2
4. Dettmers, Pagnoni, Holtzman and Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," 2023. arXiv:2305.14314v1
5. Dettmers and Zettlemoyer, "The Case for 4-bit Precision: k-bit Inference Scaling Laws," ICML 2023. arXiv:2212.09720v2
6. Frantar, Ashkboos, Hoefler and Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," ICLR 2023. arXiv:2210.17323v2
7. Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," MLSys 2024. arXiv:2306.00978v6
8. Merity, Xiong, Bradbury and Socher, "Pointer Sentinel Mixture Models" (WikiText), 2016. arXiv:1609.07843v1
9. FitzGerald et al., "MASSIVE: A 1M-Example Multilingual Natural Language Understanding Dataset with 51 Typologically-Diverse Languages," ACL 2023. arXiv:2204.08582v2

References (cont.)

10. Yu, Jeong, Kim, Kim and Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI 2022. usenix.org
11. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP 2023. arXiv:2309.06180v1 (source of Figure 3)
12. Rajbhandari, Rasley, Ruwase and He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. arXiv:1910.02054v3
13. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022. arXiv:2106.09685v2
14. GGML, "GGUF" file format specification. github.com/ggml-org/ggml
15. Hugging Face, "GGUF" (Hub documentation, quantization types). huggingface.co/docs/hub/gguf
16. `llama.cpp`, quantize README. github.com/ggml-org/llama.cpp

References (cont.)

17. Ollama documentation: OpenAI compatibility, Modelfile, importing a model. docs.ollama.com; model tags, ollama.com/library/qwen3/tags
18. vLLM documentation: engine arguments, automatic prefix caching, installation, online serving. docs.vllm.ai
19. Hugging Face, "bitsandbytes" quantization guide. huggingface.co/docs/transformers
20. Hugging Face PEFT documentation, LoRA. huggingface.co/docs/peft
21. NVIDIA, "T4 Tensor Core GPU" (16 GB GDDR6, 320+ GB/s). nvidia.com
22. Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," ICLR 2017. arXiv:1701.06538v1
23. Fedus, Zoph and Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," JMLR 2022. arXiv:2101.03961v3
24. Jiang et al., "Mixtral of Experts," 2024. arXiv:2401.04088v1
25. DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024. arXiv:2412.19437v2

References (cont.)

26. OpenAI, "gpt-oss-120b & gpt-oss-20b Model Card," 2025. arXiv:2508.10925v1; huggingface.co/openai/gpt-oss-20b
27. Muennighoff et al., "OLMoE: Open Mixture-of-Experts Language Models," ICLR 2025. arXiv:2409.02060v2
28. `llama.cpp`, server README (`--cpu-moe`, `--n-cpu-moe`). github.com/ggml-org/llama.cpp
29. Leviathan, Kalman and Matias, "Fast Inference from Transformers via Speculative Decoding," ICML 2023. arXiv:2211.17192v2
30. Chen et al., "Accelerating Large Language Model Decoding with Speculative Sampling," 2023. arXiv:2302.01318v1
31. Saxena, "Prompt Lookup Decoding," 2023. github.com/apoorvumang/prompt-lookup-decoding
32. Hugging Face `transformers` documentation: assisted decoding, KV cache (prefix caching), streamers. huggingface.co/docs/transformers
33. vLLM documentation, "Speculative Decoding." docs.vllm.ai